```
PHP Notice:  A non well formed numeric value encountered
int(101)
PHP Warning:  A non-numeric value encountered
int(1)

Output: (PHP 8.4)
bool(true)
bool(false)
int(124)
float(123.1)
Warning: A non-numeric value encountered
int(101)
Fatal error: Uncaught TypeError: Unsupported operand types: string + int
```

The first two lines demonstrate what PHP considers a numeric string to be, using its built-in `is_numeric()` function. We then use those strings in various operations: adding an integer to `"123"` means the string behaves like an integer, and adding a float coerces it to a float.

Things get more interesting when we use `"100 Dalmatians"`: despite the fact that it is not strictly numeric, it does contain the number *100*, and, for better or worse, PHP allows that number to be extracted and used in numerical expressions, as we can see from the output. We're able to take the value *100* from the string and add a number to it, returning an integer value of `101`. Granted, we get a notice message when that happens, but the code continues to execute.

We're also able, with PHP 7.4, to use a numberless string as a number in the expression `"Five-hundred."` In this case, the string is cast to the integer `0`. This particular feature of PHP has led to issues in the past: in PHP 7.4 and earlier, the expression `"a string" == 0` evaluates to `true`; combine this with the not-unusual practice of using integers `1` and `0` as substitutes for booleans, and we have a potential recipe for logical errors. PHP 8, however, has done away with this oddity; `"a string" == 0` now evaluates to `false`, and the expression `"Five-hundred" + 1` is illegal and emits a `TypeError`.

There's a lot more to say on the topic of type coercion in PHP, and in particular the edge cases and weird behavior to watch out for, which we'll review in this chapter and again where relevant.